Boosting Methods — English Notes

Nov 8, 2026 · @Razeen

Comprehensive notes on Boosting Methods for the ML course. Covers all major boosting algorithms with theory, formulas, and implementation.

1. Introduction to Boosting

Ensemble Learning combines multiple models to build a more accurate predictor than any single model alone. Like a panel of expert doctors reaching a diagnosis together, ensemble models outperform individual ones.

Bagging vs Boosting

Feature	Bagging	Boosting
Training Style	Parallel (all at once)	Sequential (one after another)
Error Focus	Random sampling	Focuses on previous model's errors
Example Algorithm	Random Forest	AdaBoost, GBM
Overfitting Risk	Lower	Slightly higher
Speed	Faster	Slower (sequential)
Primary Fix	High variance	High bias

Weak Learner vs Strong Learner

Weak Learner: A model only slightly better than random guessing. The classic example is a **Decision Stump** — a decision tree with max_depth=1 that asks only one question.

Strong Learner: An ensemble of many weak learners that achieves high accuracy.

Core Principle of Boosting: Train weak learners sequentially. Each new learner concentrates on the samples that previous learners got wrong.

Intuition

Imagine a student repeatedly taking practice exams. The first tutor teaches fundamentals. The second focuses on the topics where the student failed. The third drills the hardest questions. Each iteration targets the remaining weaknesses — exactly how boosting works.

2. Boosting Methods Overview

Several major boosting algorithms are used in industry. Each builds on the same core idea but with different optimizations.

Algorithm	Year	Key Innovation	Best For
AdaBoost	1997	Adaptive sample weighting	Small-medium datasets, clean data
Gradient Boosting (GBM)	1999	Gradient descent on residuals	Medium datasets, flexible loss
XGBoost	2014	Regularization + parallel trees	Kaggle, production systems
LightGBM	2017	Leaf-wise growth, histogram binning	Very large datasets
CatBoost	2017	Automatic categorical feature handling	Mixed data types

Industry Applications

- **AdaBoost:** Medical diagnosis, face detection (Viola-Jones algorithm used in cameras)
- **GBM:** Credit risk scoring, insurance pricing models
- **XGBoost:** Kaggle competition winner for years, fintech fraud detection
- **LightGBM:** Real-time recommendation systems (Netflix, Amazon)
- **CatBoost:** E-commerce click-through rate prediction (Yandex)

3. AdaBoost Theory

AdaBoost (Adaptive Boosting) was introduced by Freund and Schapire in 1997. It won the Godel Prize for its theoretical contributions.

Algorithm Steps

1. Initialize equal sample weights: w_i = 1/N for all i
2. Train a weak learner h_t on the weighted data
3. Compute weighted error rate:

```
\varepsilon_t = \frac{\sum_{i=1}^{N} w_i \cdot \mathbb{1}[y_i \neq h_t(x_i)]}
{\sum_{i=1}^{N} w_i}
```

4. Compute the learner's weight (importance):

```
\alpha_t = \frac{1}{2} \ln\left(\frac{1 - \varepsilon_t}
{\varepsilon_t}\right)
```

5. Update sample weights — misclassified samples get higher weight:

```
w_i^{(t+1)} = w_i^{(t)} \cdot \exp(-\alpha_t \cdot y_i \cdot h_t(x_i))
```

6. Normalize weights so they sum to 1
7. Repeat steps 2-6 for T rounds
8. Final prediction (weighted majority vote):

```
H(x) = \text{sign}\left(\sum_{t=1}^{T} \alpha_t \cdot h_t(x)\right)
```

Understanding Alpha

Scenario	Alpha Value	Meaning
Error near 0	Large positive	Near-perfect learner, high influence
Error = 0.5	0	Random guessing, no contribution
Error near 1	Large negative	Intentionally wrong (flip predictions)

Decision Stump: A tree with max_depth=1. It evaluates one threshold on one feature: the simplest possible classifier and AdaBoost's default base estimator.

4. AdaBoost Implementation (sklearn)

Core Code

```
from sklearn.ensemble import AdaBoostClassifier
from sklearn.tree import DecisionTreeClassifier

ada = AdaBoostClassifier(
    estimator=DecisionTreeClassifier(max_depth=1),  # Decision Stump
    n_estimators=100,     # Number of weak learners
    learning_rate=1.0,    # Contribution of each learner
    algorithm='SAMME',    # Discrete AdaBoost (sklearn 1.4+)
    random_state=42
)
ada.fit(X_train, y_train)
```

Key Hyperparameters

Parameter	Default	Description
n_estimators	50	Number of boosting rounds
learning_rate	1.0	Shrinks each learner's contribution
estimator	DecisionTreeClassifier(max_depth=1)	Base weak learner
algorithm	'SAMME'	Discrete boosting (SAMME.R deprecated)

Evaluation Methods

```
# Old way: manual confusion matrix
from sklearn.metrics import confusion_matrix
import matplotlib.pyplot as plt
cm = confusion_matrix(y_test, ada.predict(X_test))
plt.imshow(cm, cmap='Blues')

# New way: one line
from sklearn.metrics import ConfusionMatrixDisplay
ConfusionMatrixDisplay.from_estimator(ada, X_test, y_test, cmap='Blues')
plt.show()

# Track accuracy round by round
train_scores = list(ada.staged_score(X_train, y_train))
test_scores = list(ada.staged_score(X_test, y_test))
```

Common Pitfalls

- Using `algorithm='SAMME.R'` — deprecated in sklearn 1.4+, use `'SAMME'`
- Too many estimators without tuning learning_rate can overfit
- AdaBoost is sensitive to noisy/outlier data (they get high weights)

5. Gradient Boosting Theory

Gradient Boosting trains each new tree on the **residuals** (prediction errors) of the current ensemble. It frames boosting as gradient descent in function space.

Core Idea

Given predictions from the current model F(x), the residual for each sample is:

```
r_i = y_i - F(x_i)
```

Each new tree h_m(x) is trained to predict these residuals. The model is then updated:

```
F_m(x) = F_{m-1}(x) + \nu \cdot h_m(x)
```

where v is the **learning rate** (shrinkage factor, 0 < v <= 1).

General Formulation

Initial model (constant prediction, e.g., mean for regression):

```
F_0(x) = \arg\min_{\gamma} \sum_{i=1}^{N} L(y_i, \gamma)
```

Pseudo-residuals at each step (gradient of the loss):

```
r_{im} = -\left[\frac{\partial L(y_i, F(x_i))}{\partial
F(x_i)}\right]_{F=F_{m-1}}
```

For MSE loss this simplifies to: r_i = y_i - F(x_i)

Key Parameters and Their Effect

Parameter	Effect	Recommendation
learning_rate (v)	Lower = more conservative updates	0.01-0.3
n_estimators	More trees = fits residuals more	100-500
max_depth	Deeper = more complex interactions	3-5
subsample	Fraction of data per tree (Stochastic GB)	0.7-0.9
max_features	Features per split	'sqrt' or 0.5-0.8

Key Tradeoff: Lower learning_rate + more trees = better generalization, but slower training. A common starting point: learning_rate=0.1, n_estimators=100.

6. Gradient Boosting Implementation (sklearn)

Core Code

```
from sklearn.ensemble import GradientBoostingClassifier

gbm = GradientBoostingClassifier(
    n_estimators=100,
    learning_rate=0.1,
    max_depth=3,
    subsample=0.8,       # Stochastic GB: 80% of data per tree
    max_features='sqrt', # Random subspace method
    random_state=42
)
gbm.fit(X_train, y_train)
```

AdaBoost vs Gradient Boosting

Feature	AdaBoost	Gradient Boosting
What is updated	Sample weights	Residuals (pseudo-gradients)
Loss function	Exponential	Any differentiable loss
Noise sensitivity	High	Moderate
Overfitting	More resistant	More prone (needs tuning)
Flexibility	Less	More
Typical depth	1 (stumps)	3-5

Feature Importance and Evaluation

```
import pandas as pd

# Feature importance
feat_imp = pd.Series(gbm.feature_importances_, index=X.columns)
feat_imp.nlargest(10).sort_values().plot(kind='barh')

# Learning curve
train_scores = list(gbm.staged_score(X_train, y_train))
test_scores = list(gbm.staged_score(X_test, y_test))
plt.plot(train_scores, label='Train')
plt.plot(test_scores, label='Test')
plt.xlabel('n_estimators')
plt.legend()
```

Stochastic Gradient Boosting

Setting `subsample < 1.0` creates **Stochastic Gradient Boosting** — each tree sees only a random fraction of the data. This:

- Reduces overfitting
- Speeds up training
- Often improves generalization

7. Hyperparameter Tuning with GridSearchCV

```
from sklearn.model_selection import GridSearchCV
from sklearn.ensemble import GradientBoostingClassifier

param_grid = {
    'n_estimators': [50, 100, 200],
    'learning_rate': [0.05, 0.1, 0.2],
    'max_depth': [2, 3, 4]
}

grid = GridSearchCV(
    GradientBoostingClassifier(subsample=0.8, random_state=42),
    param_grid=param_grid,
    cv=5,
    scoring='accuracy',
```

```
    n_jobs=-1   # Use all CPU cores
)
grid.fit(X_train, y_train)

print('Best params:', grid.best_params_)
print('Best CV score:', grid.best_score_)
print('Test score:', grid.best_estimator_.score(X_test, y_test))
```

Tuning Guidelines

Parameter	Range to Search	Notes
n_estimators	50, 100, 200, 500	More is usually better with low lr
learning_rate	0.01, 0.05, 0.1, 0.2	Lower = needs more estimators
max_depth	2, 3, 4, 5	Start with 3; deeper = more complex
subsample	0.7, 0.8, 0.9	Stochastic GB regularization
min_samples_split	2, 5, 10, 20	Higher = prevents overfitting

Rule of thumb: Fix n_estimators=100, tune learning_rate and max_depth first. Then increase n_estimators if more performance is needed.

8. Real World Applications + Summary

Real World Use Cases

Industry	Use Case	Preferred Algorithm
Healthcare	Cancer/disease risk classification	AdaBoost, GBM
Banking	Credit scoring, loan default prediction	XGBoost, GBM
Insurance	Claim frequency and severity prediction	GBM, XGBoost
E-commerce	Click-through rate, product recommendations	LightGBM
HR Analytics	Employee attrition, performance prediction	AdaBoost

Industry	Use Case	Preferred Algorithm
Search	Ranking search results (LambdaMART)	GBM variant
Autonomous Vehicles	Object detection pipelines	AdaBoost (Viola-Jones)

Complete Reference: All Key Concepts

Concept	Definition	Formula / Code
Weak Learner	Model slightly better than random	DecisionTreeClassifier(max_depth=1)
Ensemble	Combination of multiple models	Weighted sum of predictions
AdaBoost Error	Weighted misclassification rate	e_t = sum(w_i * wrong_i) / sum(w_i)
AdaBoost Alpha	Learner's weight in final vote	0.5 * ln((1-e) / e)
Weight Update	Increase weights for hard samples	w *= exp(-alpha * y * pred)
GBM Residual	Prediction error	r_i = y_i - F(x_i)
GBM Update Rule	Add new tree's correction	F_m = F_{m-1} + lr * h_m
Learning Rate	Shrinkage / contribution control	Usually 0.01 to 0.3
Staged Score	Per-round accuracy	model.staged_score(X, y)
Feature Importance	Each feature's contribution	model.feature_importances_
Stochastic GB	Random subsampling per tree	subsample=0.8

What to Learn Next

1. **XGBoost** — `pip install xgboost`; same sklearn API, adds L1/L2 regularization
2. **LightGBM** — Microsoft's fast GBM for large datasets
3. **CatBoost** — Yandex's GBM with native categorical feature support

4. **SHAP Values** — Model-agnostic feature attribution for explainability
5. **Stacking / Blending** — Combining diverse models with a meta-learner